NS 1. XOR Operation on String

Initialize a string (e.g., "Hello World")

Traverse each character of the string

Perform XOR operation with 0 for each character

Store the result back in the string

Display the modified string , Stop

Code

```
#include <stdio.h>  int main(){
char str[]="Hello World";  int i=0;
while(str[i]!='\0'){ str[i]=str[i]^0; i++; }
printf("Result: %s",str);  return 0; }
```

2. AND, OR, XOR with 127

Initialize string "Hello World"

Traverse each character

Perform: AND with 127

OR with 127, XOR with 127

Print results separately , Stop

Code

```
#include <stdio.h> int main(){
char str[]="Hello World";  int i;
printf("AND Operation:\n");
for(i=0;str[i]!='\0';i++)
printf("%c",str[i]&127);
printf("\nOR Operation:\n");
for(i=0;str[i]!='\0';i++)
printf("%c",str[i]|127);
printf("\nXOR Operation:\n");
for(i=0;str[i]!='\0';i++)
printf("%c",str[i]^127);  return 0;  }
```

3)encryption and decryption

a)caeser cipher

Input string and key , For each character:

Encryption: (char + key) % 26

Decryption: (char - key) % 26

Preserve uppercase/lowercase

Display encrypted and decrypted text stop

Code

```
import java.util.*;  class Caesar{
public static void main(String args[]){
Scanner sc=new Scanner(System.in);
String text;  int key,i;
System.out.print("Enter text: ");
text=sc.nextLine();
System.out.print("Enter key: ");
key=sc.nextInt();
char[] ch=text.toCharArray();
for(i=0;i<ch.length;i++){
if(Character.isLetter(ch[i])){  char
base=Character.isUpperCase(ch[i])?'A':'a';
ch[i]=(char)((ch[i]-base+key)%26+base);}}
System.out.println("Encrypted:
"+String.valueOf(ch));
for(i=0;i<ch.length;i++){
if(Character.isLetter(ch[i])){ char
base=Character.isUpperCase(ch[i])?'A':'a';
ch[i]=(char)((ch[i]-base-
key+26)%26+base);}}
System.out.println("Decrypted:
"+String.valueOf(ch));}}
```

3b) Substitution Cipher

Define mapping of alphabets

Input plaintext

Replace each character using mapping

Display ciphertext Stop

Code import java.util.*; class Substitution{

public static void main(String args[]){

String normal="abcdefghijklmnopqrstuvwxyz";

String cipher="qwertyuiopasdfghjklzxcvbnm";

Scanner sc=new Scanner(System.in);

String text,res="";

System.out.print("Enter text: ");

text=sc.nextLine();

for(int i=0;i<text.length();i++){

char ch=text.charAt(i);

int index=normal.indexOf(ch);

if(index!=-1) res+=cipher.charAt(index);

else res+=ch; }

System.out.println("Encrypted: "+res);}}

3c) hill cipher

Take plaintext and key matrix (2x2)

Convert text into numbers (A=0…Z=25)

Multiply matrix with text vector

Take modulo 26

Convert numbers back to characters

Display ciphertext

Code import java.util.*; class Hill{

public static void main(String args[]){

int key[][]={{3,3},{2,5}}; String text="HI";

int a=text.charAt(0)-'A';

int b=text.charAt(1)-'A';

int c1=(key[0][0]*a+key[0][1]*b)%26;

int c2=(key[1][0]*a+key[1][1]*b)%26;

System.out.println("Encrypted: "+(char)(c1+'A')+(char)(c2+'A'));}}

4)DES Import crypto packages

Define plaintext and key

Convert key into SecretKeySpec

Create Cipher object for DES

Initialize cipher in ENCRYPT mode

Encrypt data, Initialize cipher in DECRYPT mode,Decrypt data,Display results

Code import javax.crypto.Cipher;

import javax.crypto.spec.SecretKeySpec;

class DES{

public static void main(String args[]) throws Exception{ String key="12345678";

String text="HelloDES";

SecretKeySpec sk=new SecretKeySpec(key.getBytes(),"DES");

Cipher c=Cipher.getInstance("DES");

c.init(Cipher.ENCRYPT_MODE,sk);

byte[] enc=c.doFinal(text.getBytes());

System.out.println("Encrypted: "+new String(enc));

c.init(Cipher.DECRYPT_MODE,sk);

byte[] dec=c.doFinal(enc);

System.out.println("Decrypted: "+new String(dec));}}

5)blowfish

Import crypto packages

Define plaintext and key

Create SecretKeySpec using Blowfish algorithm

Get Cipher instance for Blowfish

Initialize cipher in ENCRYPT mode

Encrypt the plaintext

Initialize cipher in DECRYPT mode

Decrypt the ciphertext

Display encrypted and decrypted data

Code

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
class Blowfish{
public static void main(String args[]) throws Exception{
String key="MyBlowKey";
String text="HelloBlowfish";
SecretKeySpec sk=new SecretKeySpec(key.getBytes(),"Blowfish");
Cipher c=Cipher.getInstance("Blowfish");
c.init(Cipher.ENCRYPT_MODE,sk);
byte[] enc=c.doFinal(text.getBytes());
System.out.println("Encrypted: "+new String(enc));
c.init(Cipher.DECRYPT_MODE,sk);
byte[] dec=c.doFinal(enc);
System.out.println("Decrypted: "+new String(dec));}}
```

6)Rijndael (AES)

Define plaintext and 16-byte key

Create SecretKeySpec

Get AES Cipher instance

Initialize for encryption

Encrypt data

Initialize for decryption

Decrypt data

Display results

Code

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
class AES{ public static void main(String args[]) throws Exception{
String key="1234567812345678";
String text="HelloAES";
SecretKeySpec sk=new SecretKeySpec(key.getBytes(),"AES");
Cipher c=Cipher.getInstance("AES");
c.init(Cipher.ENCRYPT_MODE,sk);
byte[] enc=c.doFinal(text.getBytes());
System.out.println("Encrypted: "+new String(enc));
c.init(Cipher.DECRYPT_MODE,sk);
byte[] dec=c.doFinal(enc);
System.out.println("Decrypted: "+new String(dec));}}
```

CD

1)identify diff types of tokens

Read a line of C program input

Initialize variables to store tokens

Define a list of all C keywords

Traverse the input string character by character, If the character is alphanumeric, keep forming a token

When a delimiter or space is found:

Terminate the token, Check whether it is a keyword, identifier, or constant

If the character is a special symbol:

Check whether it is an operator

Print each token with its type

Continue until end of string

Code #include<stdio.h>#include<string.h>

#include<ctype.h> int isKeyword(char *str){ char *kw[]={"auto","break","case","char","const","continue","default","do","double","else","enum","extern","float","for","goto","if","int","long","register","return","short","signed","sizeof","static","struct","switch","typedef","union","unsigned","void","volatile","while"}; for(int i=0;i<32;i++)

if(strcmp(str,kw[i])==0) return 1;return 0;}

int main(){ char str[200],tok[50];

int i=0,j=0;- gets(str); while(str[i]!='\0'){

if(isalnum(str[i])) tok[j++]=str[i]; else{

tok[j]='\0'; if(j!=0){

if(isKeyword(tok)) printf("%s Keyword\n",tok);

else if(isdigit(tok[0])) printf("%s Constant\n",tok);

else printf("%s Identifier\n",tok);

j=0; } if(strchr("+-*/=%<>!",str[i])) printf("%c Operator\n",str[i]);}i++;}}

2)lexical analyzer

Read the input string

Initialize index pointer to scan the string

Traverse the string character by character

If a letter is found: Continue reading until alphanumeric sequence ends

Print it as identifier, If a digit is found:

Continue reading digits, Print it as number

If an operator symbol is found:

Print it as operator

Ignore spaces and newline characters

Repeat until end of string

Code #include<stdio.h> #include<ctype.h>

#include<string.h> int main(){

char str[200]; int i=0; gets(str); while(str[i]!='\0'){ if(isalpha(str[i])){

while(isalnum(str[i])) printf("%c",str[i++]);

printf(" Identifier\n");}

else if(isdigit(str[i])){

while(isdigit(str[i])) printf("%c",str[i++]);

printf(" Number\n");}

else if(strchr("+-*/=",str[i])){

printf("%c Operator\n",str[i]);

i++;}else i++;}

}

3)brute force parsing

Define the grammar, Read the input string

Calculate the length of the string

Check if the string follows the pattern:

First half contains 'a',

Second half contains 'b'

Compare characters from start and end simultaneously

If mismatch occurs, mark as invalid

Ensure total length is even

If all conditions satisfied → Accept

Otherwise → Reject

Code

```
#include<stdio.h> #include<string.h>

int main(){ char str[20]; scanf("%s",str);

int len=strlen(str); int flag=1;

for(int i=0;i<len/2;i++){

if(str[i]!='a' || str[len-i-1]!='b'){

flag=0; break; } }

if(flag && len%2==0) printf("Accepted");

else printf("Rejected");}
```

4)recursive descent parser

Define grammar rules (E → T E', E' → + T E' | ε, T → a) , Read input string

Initialize pointer to first character

Call function E() as starting symbol

In E(): Call T(),Then call E'(),In E'():

If '+' is found:, Move pointer forward

Call T() and E'() again

Else return (ε production)

In T():, If 'a' is found, move pointer

Else report error

After parsing, check if entire string is consumed , If yes → Accept, else Reject

Code

```
#include<stdio.h> char str[20]; int i=0;

void E(); void E1(); void T(); void E(){ T();

E1(); } void E1(){ if(str[i]=='+'){ i++; T();

E1(); } } void T(){ if(str[i]=='a') i++;

else{ printf("Rejected"); exit(0);}}

int main(){ scanf("%s",str); E();

if(str[i]=='\0') printf("Accepted");

else printf("Rejected");}
```

5)first and follow

Read number of productions and grammar rules

Identify terminals and non-terminals

For FIRST set:

If symbol is terminal, add directly

If non-terminal:

Recursively check its productions

Add first symbol of each production

For FOLLOW set: Add '$' to start symbol

For each occurrence of symbol in productions:

If followed by terminal → add it

If followed by non-terminal → add its FIRST, If at end → add FOLLOW of LHS

Repeat until no new elements are added

Display FIRST and FOLLOW sets

9)shift reduce parser

Initialize an empty stack

Read input string and append end marker

Set pointer to first symbol of input

Repeat until input is fully processed:

Shift: move input symbol to stack

After each shift, check stack content

If stack matches a grammar rule (handle found), reduce it

Replace matched substring with non-terminal (e.g., E)

Continue shifting and reducing alternately

If stack contains only start symbol and input is consumed: Accept string, Else reject string

Code

```
#include<stdio.h>
#include<string.h> int main(){
char input[20],stack[20],temp[20];
int i=0,j=0; printf("Enter input string: ");
scanf("%s",input); strcat(input,"$");
printf("\nStack\tInput\tAction\n");
while(input[i]!='\0'){ stack[j++]=input[i++];
stack[j]='\0';
printf("%s\t%s\tSHIFT\n",stack,&input[i]);
if(strcmp(stack,"ab")==0){
stack[0]='E'; stack[1]='\0';j=1;
printf("%s\t%s\tREDUCE E->ab\n",stack,&input[i]);}}
if(strcmp(stack,"E")==0)
printf("\nAccepted\n");
else  printf("\nRejected\n"); return 0; }
```

10) three address code

Read arithmetic expression

Break expression into sub-expressions

Identify operators based on precedence

For each operator:

Generate temporary variable (t1, t2, ...)

Store intermediate result

Replace sub-expression with temporary variable

Continue until expression is fully converted

Display TAC format

Code

```
#include<stdio.h>   #include<string.h>
int main(){ char exp[50]; int t=1;
printf("Enter expression: ");
scanf("%s",exp);
printf("\nThree Address Code:\n");
for(int i=0;i<strlen(exp);i++){
if(exp[i]=='*' || exp[i]=='/' || exp[i]=='+' || exp[i]=='-'){
printf("t%d = %c %c %c\n",t,exp[i-1],exp[i],exp[i+1]);
exp[i+1]='t';
exp[i+1]=(char)(t+'0');
t++; } }
printf("\nFinal result stored in last temporary variable\n");
return 0; }
```

7)RSA

Generate public and private keys

Create Cipher object

Initialize with public key for encryption

Encrypt plaintext

Initialize with private key for decryption

Decrypt ciphertext, Display results

Code

```
import javax.crypto.Cipher;

import java.security.*;

class RSA{

public static void main(String args[])
throws Exception{

KeyPairGenerator
kpg=KeyPairGenerator.getInstance("RSA");

kpg.initialize(512);

KeyPair kp=kpg.generateKeyPair();

PublicKey pub=kp.getPublic();

PrivateKey pri=kp.getPrivate();

Cipher c=Cipher.getInstance("RSA");

String text="HelloRSA";

c.init(Cipher.ENCRYPT_MODE,pub);

byte[] enc=c.doFinal(text.getBytes());

System.out.println("Encrypted: "+new
String(enc));

c.init(Cipher.DECRYPT_MODE,pri);

byte[] dec=c.doFinal(enc);

System.out.println("Decrypted: "+new
String(dec));}}
```

8)SHA Import required security package

Create Scanner object to read input

Read input message from user

Create MessageDigest instance for SHA-1

Convert input string into byte array

Pass byte array to digest() method

Store resulting hash bytes

Convert each byte into hexadecimal format, Concatenate all hex values

Display final hash value

Code

```
import java.util.*;  import java.security.*;

class SHA1{

public static void main(String args[])
throws Exception{

Scanner sc=new Scanner(System.in);

System.out.print("Enter message: ");

String text=sc.nextLine();

MessageDigest
md=MessageDigest.getInstance("SHA-1");

byte[] input=text.getBytes();

md.update(input);

byte[] hash=md.digest();

StringBuffer hex=new StringBuffer();

for(int i=0;i<hash.length;i++){

int val=hash[i]&0xff;

if(val<16)  hex.append("0");

hex.append(Integer.toHexString(val));}

System.out.println("SHA-1 Hash:
"+hex.toString());}}
```

9)diffie hellman

Choose two public numbers:

Prime number p, Base (primitive root) g

User A selects a private key a

User B selects a private key b

Compute public values:

A = g^a mod p, B = g^b mod p

Exchange A and B

Compute shared secret:

User A: key = B^a mod p

User B: key = A^b mod p

Both keys will be equal , Display results

Code <!DOCTYPE html><html><head>

<title>Diffie Hellman</title></head>

<body> <h2>Diffie-Hellman Key Exchange</h2> Enter Prime (p): <input type="number" id="p">

Enter Base (g): <input type="number" id="g">

Enter Private Key of A: <input type="number" id="a">

Enter Private Key of B: <input type="number" id="b">

<button onclick="calculate()">Generate Key</button> <h3 id="result"></h3>

<script> function power(base,exp,mod){

let result=1; for(let i=0;i<exp;i++)

result=(result*base)%mod; return result;}

function calculate(){ let p=parseInt(document.getElementById("p").value);

letg=parseInt(document.getElementById("g").value); let a=parseInt(document.getElementById("a").value); let b=parseInt(document.getElementById("b").value); let A=power(g,a,p); let B=power(g,b,p); let keyA=power(B,a,p);

let keyB=power(A,b,p);

document.getElementById("result").innerHTML= "Public Key A: "+A+"<br>"+

"Public Key B: "+B+"<br>"+

"Secret Key (A): "+keyA+"<br>"+

"Secret Key (B): "+keyB;}</script></body>

</html>

10)MD5 Input the message (plain text)

Convert the message into bytes

Initialize MD5 message digest object

Update the digest with input bytes

Compute the hash value (128-bit)

Convert the byte array into hexadecimal format Display the final hash value

Code import java.security.*;

class MD5{ public static void main(String args[]) throws Exception{

String text="HelloMD5"; MessageDigest md=MessageDigest.getInstance("MD5");

md.update(text.getBytes());

byte[] hash=md.digest();

System.out.print("MD5 Hash: ");

for(int i=0;i<hash.length;i++){

System.out.print(Integer.toHexString((hash[i] & 0xff) | 0x100).substring(1));}}}

11)RC4

Take plaintext and secret key as input

Initialize two arrays:

S[0…255] with values 0 to 255

K[0…255] using the key (repeat key if needed)

Perform Key Scheduling Algorithm (KSA):

Initialize j = 0

For i = 0 to 255:

j = (j + S[i] + K[i]) mod 256

Swap S[i] and S[j]

Perform Pseudo Random Generation Algorithm (PRGA):

Initialize i = 0, j = 0

For each character in plaintext:

i = (i + 1) mod 256

j = (j + S[i]) mod 256

Swap S[i] and S[j]

t = (S[i] + S[j]) mod 256

KeyStream = S[t]

CipherText = PlainText XOR KeyStream

Display ciphertext

For decryption, apply same process (RC4 is symmetric)

Code

```
import java.util.*;

class RC4{

public static void main(String args[]){

String key="KEY";

String text="HELLO";

int S[]=new int[256];

int K[]=new int[256];

for(int i=0;i<256;i++){

S[i]=i;  K[i]=key.charAt(i%key.length());}

int j=0; for(int i=0;i<256;i++){

j=(j+S[i]+K[i])%256; int temp=S[i]; S[i]=S[j];

S[j]=temp;} int i=0;  j=0;  String cipher="";

for(int n=0;n<text.length();n++){

i=(i+1)%256;j=(j+S[i])%256;int temp=S[i];

S[i]=S[j]; S[j]=temp;  int t=(S[i]+S[j])%256;

int k=S[t];

char c=(char)(text.charAt(n)^k);

cipher+=c;}

System.out.println("Cipher Text: "+cipher);

}}
```

7)predictive parser

Read input string and grammar

Initialize stack with '$' and start symbol

Set pointer to first input symbol

Repeat until stack becomes empty:

Compare top of stack with input symbol

If both match:

Pop stack and move input pointer

Else if top is non-terminal:

Replace using parsing table rule

Else: Report error,

If both input and stack reach '$':

Accept string, Otherwise reject

Code

```
#include<stdio.h> #include<string.h>
char stack[20]; int top=-1;
void push(char c){ stack[++top]=c; }
char pop(){ return stack[top--];}
int main(){ char input[20];
scanf("%s",input); push('$'); push('E');
int i=0; while(top!=-1){ char t=pop();
if(t==input[i]) i++; else if(t=='E'){
push('a'); }
else{ printf("Rejected");
return 0;} }
if(input[i]=='\0') printf("Accepted");
else printf("Rejected");}
```

8)LR parsing

Read input string

Initialize stack with initial state

Set input pointer to first symbol

Repeat: Check top of stack and current input

Use parsing table to decide action

If action is SHIFT:

Push symbol to stack

Move input pointer, If action is REDUCE:

Replace symbols with non-terminal

If action is ACCEPT:, Print success message

If no valid action: Report error

Code

```
#include<stdio.h>  #include<string.h>
int main(){ char input[20],stack[20];
int i=0,j=0;
scanf("%s",input);
while(input[i]!='\0'){
stack[j++]=input[i++];
stack[j]='\0';
if(strcmp(stack,"ab")==0){
stack[0]='S';
stack[1]='\0';
j=1;}}
if(strcmp(stack,"S")==0)
printf("Accepted");
else printf("Rejected");}
```

First and follow code

```
#include<stdio.h>
#include<string.h>char prod[10][10];

char first[10],follow[10]; int n;

void findFirst(char c){ for(int i=0;i<n;i++){

if(prod[i][0]==c){ if(!isupper(prod[i][3]))
printf("%c ",prod[i][3]);
else findFirst(prod[i][3]);}}}
void findFollow(char c){

if(prod[0][0]==c) printf("$ ");

for(int i=0;i<n;i++){

for(int j=3;j<strlen(prod[i]);j++){

if(prod[i][j]==c){  if(prod[i][j+1]!='\0')

findFirst(prod[i][j+1]);  else if(c!=prod[i][0])

findFollow(prod[i][0]);}}}}  int main(){

printf("Enter number of productions: ");

scanf("%d",&n);

printf("Enter productions (E->E+T):\n");

for(int i=0;i<n;i++) scanf("%s",prod[i]);

char c;   printf("Enter symbol: ");

scanf(" %c",&c); printf("FIRST: ");

findFirst(c);  printf("\nFOLLOW: ");

findFollow(c); }
```

6)left recursion and left factoring

Read grammar production

Check for left recursion:

If production is of form A → Aα | β

Separate recursive and non-recursive parts, Create new non-terminal A'

Rewrite grammar as: A → βA', A' → αA' | ε
For left factoring:
Find common prefix among productions

Factor out common part, Introduce new non-terminal,Display transformed grammar

Code

```
#include<stdio.h> #include<string.h>

int main(){ char prod1[50],prod2[50];

char lhs,alpha[20],beta[20],prefix[20];

int i,j; printf("Enter production for Left Recursion (e.g. E->E+T|T): ");

scanf("%s",prod1);  printf("Enter production for Left Factoring (e.g. E->iEt|iEs): "); scanf("%s",prod2);

\\left recursion

lhs=prod1[0]; if(prod1[3]==lhs){ i=4; j=0;

while(prod1[i]!='|') alpha[j++]=prod1[i++];

alpha[j]='\0';i++;j=0;while( prod1[i]!='\0')

beta[j++]=prod1[i++];beta[j]='\0';

printf("\nLeft Recursion Eliminated:\n");

printf("%c->%s%c'\n",lhs,beta,lhs);

printf("%c'->%s%c'|e\n",lhs,alpha,lhs);}

else{ printf("\nNo Left Recursion found in first production\n");}

\\left factoring

lhs=prod2[0];i=3;j=0;  while(prod2[i]!='|' && prod2[i]==prod2[i+4]){

prefix[j++]=prod2[i];i++;}prefix[j]='\0';

if(strlen(prefix)>0){

printf("\nLeft Factoring Applied:\n");

printf("%c->%s%c'\n",lhs,prefix,lhs);

printf("%c'->remaining_productions\n",lhs);}

else{ printf("\nNo Left Factoring needed in second production\n");}return 0;}
```

11)code generation

Read three address code or expression

Initialize register counters

For each operation:

Load operands into registers

Perform operation using instruction set

Store result in register/memory

Map each TAC instruction into assembly-like code

Reuse registers when possible

Print final target code

Code

```
#include<stdio.h>

#include<string.h>

int main(){

char op[10]; int t=1;

printf("Enter simple expression (a+b*c): ");

scanf("%s",op);

printf("\nTarget Code:\n");

for(int i=0;i<strlen(op);i++){

if(op[i]=='+'){

printf("MOV R1, %c\n",op[i-1]);

printf("ADD R1, %c\n",op[i+1]);

printf("MOV R%d, R1\n",t);

t++; }    if(op[i]=='*'){

printf("MOV R2, %c\n",op[i-1]);

printf("MUL R2, %c\n",op[i+1]);

printf("MOV R%d, R2\n",t);

t++; } } return 0; }
```